

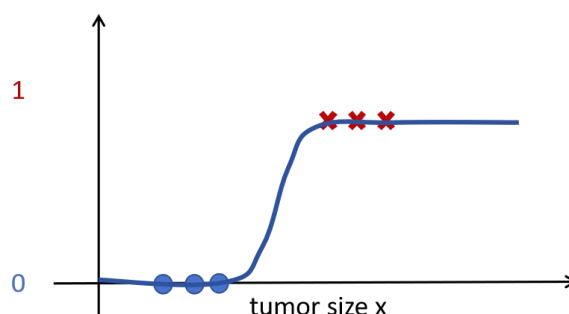
3 逻辑回归

回归问题的目标是从无数种可能的数值中预测一个数字，接下来介绍用于分类问题的模型。通常线性回归模型不适合做分类任务，因此引入另外一种模型：逻辑回归（Logistic Regression）。

垃圾邮件分类、肿瘤判断都属于分类问题，这类问题需要预测的变量只能是两个可能值之一，因此这类问题称为二分类（Binary Classification）。二分类问题中的类别可以用“no”和“yes”，“false”和“true”或 0 和 1 表示，在计算机编程中通常使用 0 和 1 表示。用“no”、“false”、0 表示的类别称为负类（Negative Class），用“yes”、“true”、1 表示的类别称为正类（Positive Class），这里的正和负并不意味着好坏，而是根据任务的目标来区分，比如任务的目标是判断一封邮件是否为垃圾邮件，那么是垃圾邮件则为正样本，不是垃圾邮件则为负样本。当然，不同人可能在实际应用中有不同的定义方式，只要训练和预测阶段前后一致即可。

3.1 逻辑回归模型

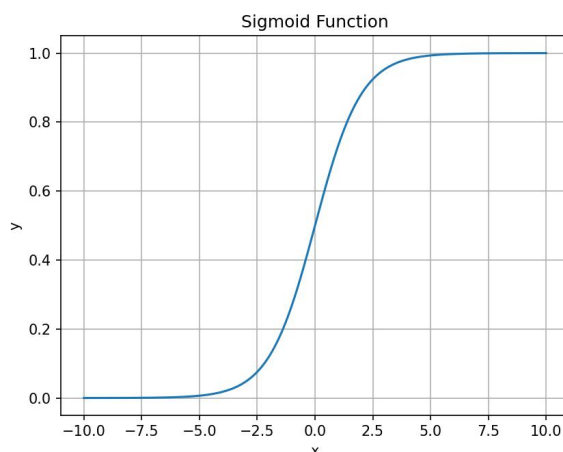
尽管逻辑回归有“回归”二字，但却是用于分类的模型，它用 sigmoid 函数估计出样本属于正样本的概率。对于一个问题，逻辑回归会拟合一条曲线，如下图所示。



sigmoid 函数（也称为 logistic 函数），sigmoid 函数定义为：

$$g(x) = \frac{1}{1 + e^{-x}}$$

sigmoid 函数的图像如下图所示：



sigmoid 函数的取值范围是 0~1，当 $x=0$ 时， $y=0.5$ 。通常 sigmoid 函数无法直接用于分类问题，比如肿瘤分类问题的输入特征 x 不可能出现负数，直接使用 sigmoid 函数的结果永远是大于等于 0.5 的。下面介绍使用 sigmoid 函数构建逻辑回归模型的步骤：

第一步：用一个线性函数将输入特征向量映射成一个实数 z ，表示为 $z = \vec{w}\vec{x} + b$ 。

第二步：将第一步的 z 作为输入给到 sigmoid 函数，表示为 $g(z) = \frac{1}{1 + e^{-z}}$ 。

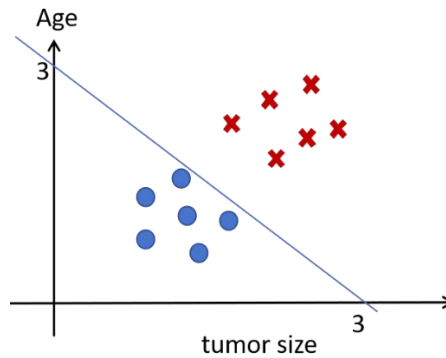
把这两步结合就得到了逻辑回归模型：

$$f_{\vec{w},b}(\vec{x}) = g(\vec{w}\vec{x} + b) = \frac{1}{1 + e^{-(\vec{w}\vec{x} + b)}}$$

逻辑回归的输入是一个或一组特征，输出一个 0~1 的实数，可以将这个输出看作是一个概率。即输入特征 x ，输出类别为正类的概率。

3.2 决策边界 (Decision Boundary)

逻辑回归模型输出一个概率，那么如何根据这个概率判断其标签是 0 还是 1？这里也需要设置一个阈值，通常设置为 0.5，当概率大于 0.5 时，预测标签是 1，否则预测标签是 0。在 sigmoid 函数中， $z=0$ 的位置函数值是 0.5，所以 $z = \vec{w}\vec{x} + b = 0$ 这条线就是决策边界。如下图所示的例子，当 $\vec{w} = [1 \ 1]$ ， $b=-3$ 时，就是我们要找的决策边界。



当参数选择不同时，决策边界也不同，因此，训练逻辑回归模型其本质是要找到合适的决策边界，而 sigmoid 函数是用来确定分类结果的。

3.3 逻辑回归中的代价函数

代价函数用来衡量模型对训练数据集的拟合程度，使模型能够尝试选择更好的参数。均方误差代价函数对于逻辑回归来说并不是一个理想的代价函数，下面介绍另一种代价函数，它可以帮助我们为逻辑回归选择更好的参数。

均方误差代价函数的公式为：

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2$$

我们可以将其改成写成如下形式：

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2$$

这里给出一个新的术语：损失函数（Loss Function）定义了单个样本预测值与真实值之间的误差。上式中求和符号里边的部分就是损失函数。损失函数用 $L(f_{\vec{w}, b}(\vec{x}), y)$ 表示，它是模型预测值和数据真实值的函数。下面不再用平方误差作为损失函数，而是换一种方法使代价函数成为一个凸函数。

$$L(f(x^{(i)}), y^{(i)}) = \begin{cases} -\ln(f(x^{(i)})), & \text{if } y^{(i)} = 1 \\ -\ln(1 - f(x^{(i)})), & \text{if } y^{(i)} = 0 \end{cases}$$

在二分类问题中，标签 y 只有 0 和 1 两个取值，损失函数可以简化为：

$$L(f(x^{(i)}), y^{(i)}) = -y^{(i)} \ln(f(x^{(i)})) - (1 - y^{(i)}) \ln(1 - f(x^{(i)}))$$

逻辑回归模型的代价函数可以写成：

$$\begin{aligned} J(\vec{w}, b) &= \frac{1}{m} \sum_{i=1}^m L(f(x^{(i)}) - y^{(i)}) \\ &= -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \ln(f(x^{(i)})) + (1 - y^{(i)}) \ln(1 - f(x^{(i)}))] \end{aligned}$$

这个损失函数是通过概率论中的最大似然估计推理出来的，这样得到的代价函数是一个凸函数，便于使用梯度下降法进行求解。

3.4 逻辑回归中的梯度下降法

使用梯度下降法求解参数 w 和 b 使代价函数 $J(\vec{w}, b)$ 最小的过程如下：

```
repeat {  
     $w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$   
     $b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$   
}
```

计算过程中最重要的部分依然是梯度的计算，在逻辑回归中梯度的计算结果如下：

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})$$

梯度的计算结果看起来与线性回归中梯度的计算结果是相同的，但实际上模型 $f_{\vec{w}, b}(\vec{x})$ 的定义是不同的。这里梯度的计算结果并不是直接沿用线性回归中的结果，而是经过推导之后刚好得到了相同的表示形式。

在线性回归中的向量化和特征缩放技巧都可以应用于逻辑回归算法中，用于提高计算效率。这些技巧在实验过程中都有体现。

Part3-1 动手练

构建一个逻辑回归模型来预测，某个学生是否被大学录取。这里通过申请学生两次测试的评分，来决定他们是否被录取。我们拥有之前申请学生的可以用于训练逻辑回归的训练样本集。对于每一个训练样本，你有他们两次测试的评分和最后是被录取的结果。

导入相关库

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

训练数据可视化

```
# 加载数据
data=pd.read_csv('ex2data1.txt',names=['x1','x2',
'admitted'])

# 可视化数据
positive = data[data['admitted'].isin([1])] # 筛选数据集中标签为
1 的数组
negative = data[data['admitted'].isin([0])] # 筛选数据集中标签为
0 的数组
plt.figure()
plt.scatter(positive['x1'], positive['x2'], s=50, c='b',
marker='o', label='admitted')
plt.scatter(negative['x1'], negative['x2'], s=50, c='r',
marker='x', label='not admitted')
plt.legend()
plt.xlabel('x1')
plt.ylabel('x2')
plt.show()
```

数据预处理

```
# Z-score 特征缩放
features = ['x1', 'x2']
data[features] = (data[features] - data[features].mean()) /
data[features].std()

# 数据预处理
data.insert(0, 'Ones', 1) # 在第 0 列插入表头为“ONE”的列，数
值为 1
cols = data.shape[1] # 获取表格 df 的列数
X = data.iloc[:,0:cols-1] # 取除最后一列外，取其他列的所有行
```

```
y = data.iloc[:,cols-1:cols]    # 取最后一列的所有行，即标签 y
X = X.values
y = y.values.flatten()
```

定义 sigmoid 函数

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```

定义代价函数

```
def compute_cost(X, y, theta):
    m = len(y)
    epsilon = 1e-15 # 防止 log(0)
    y_pred = sigmoid(np.dot(X, theta))
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    cost = -1 / m * (np.dot(y, np.log(y_pred)) + np.dot((1 - y),
np.log(1 - y_pred)))
    return cost
```

定义梯度下降法

```
def gradientDescent(X, y, theta, learning_rate=0.01,
epochs=1000):
    m, n = X.shape
    cost = np.zeros(epochs) # 初始化代价函数值为 0 数组，元素个数为
迭代次数

    for i in range(epochs):
        y_pred = sigmoid(np.dot(X, theta))
        dw = (1 / m) * np.dot(X.T, (y_pred - y))
        theta = theta - learning_rate * dw

        cost[i] = compute_cost(X, y, theta) # 计算每一次的代价函
数
    return theta, cost
```

执行梯度下降

```
# 初始化参数
theta = np.zeros(3)    # [b, w1, w2]
alpha = 0.01
epochs = 1500

# 执行梯度下降函数
theta, cost = gradientDescent(X, y, theta, alpha, epochs)
```

训练过程可视化

```
# 训练过程可视化
plt.plot(np.arange(epochs), cost, 'r')
plt.xlabel('Iterations')
plt.ylabel('Cost')
```

```
plt.title('Error vs. Training Epoch')
plt.show()
```

使用模型

```
def predict(theta, X):
    probability = sigmoid(np.dot(X, theta))
    return [1 if x >= 0.5 else 0 for x in probability] #大于
0.5 的数取值为 1, 小于 0.5 的为 0
pred_y = predict(theta, X)

accuracy = np.mean(pred_y == data['admitted'])
print(f"Accuracy: {accuracy * 100:.2f}%")
```

绘制决策边界

```
positive = data[data['admitted'].isin([1])] # 筛选数据集中标签为
1 的数组
negative = data[data['admitted'].isin([0])] # 筛选数据集中标签为
0 的数组
plt.figure()
plt.scatter(positive['x1'], positive['x2'], s=50, c='b',
marker='o', label='admitted')
plt.scatter(negative['x1'], negative['x2'], s=50, c='r',
marker='x', label='not admitted')
plt.legend()
plt.xlabel('x1')
plt.ylabel('x2')

x1 = np.arange(-2, 2, 0.01)
x2 = -theta[0]/theta[2] - theta[1] / theta[2] * x1
plt.plot(x1, x2, 'grey')
plt.title('Decision Boundary')
plt.show()
```

使用机器学习 sklearn 库实现逻辑回归算法

安装 sklearn 库: `pip install -i https://pypi.tuna.tsinghua.edu.cn/simple scikit-learn`

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 加载数据
df = pd.read_csv('ex2data1.txt', names=['x1', 'x2',
'admitted'])
X = df.iloc[:, :-1].values # 特征矩阵
```



```
y = df.iloc[:, -1].values # 目标变量

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y # 保持类别分布一致
)

# z-score 标准化特征
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 初始化逻辑回归模型
model = LogisticRegression(max_iter=1000, random_state=42)

# 训练模型
model.fit(X_train, y_train)

# 使用测试集评估模型性能
y_pred = model.predict(X_test)

print(f"测试集准确率: {accuracy_score(y_test, y_pred):.4f}")
```